

Table of contents

Dynamic Networks	1
Introduction	1
Setup	2
Static SIR Baseline	3
Dynamic SIR Model	4
Effect of Edge Dynamics	10
Inspecting Edge States	13
Dynamic SEIR	13
Simulation validation	16

Dynamic Networks

Simon Frost 2026-05-13

- [Introduction](#)
- [Setup](#)
- [Static SIR Baseline](#)
- [Dynamic SIR Model](#)
- [Effect of Edge Dynamics](#)
- [Inspecting Edge States](#)
- [Dynamic SEIR](#)
- [Simulation validation](#)

Introduction

Real contact networks are not static — edges form and break over time as individuals change partners, move between workplaces, or alter social behaviour during an epidemic. The `Dynamic-ConfigurationModel` in `EdgeBasedModels.jl` extends the static EBCM by introducing dormant edge stubs that capture this turnover.

In the dynamic model, active edges break at rate η_2 (becoming dormant) and dormant edges form at rate η_1 (becoming active). This yields an additional state variable φ_D for dormant edges. The key equations are:

$$\frac{d\theta}{dt} = - \underbrace{\beta\varphi_I}_{\text{transmission}} - \underbrace{\eta_2(\varphi_S + \varphi_I + \varphi_R)}_{\text{edge breaking}} + \underbrace{\eta_1\varphi_D}_{\text{edge forming}}$$

$$\frac{d\varphi_D}{dt} = \eta_2(\varphi_S + \varphi_I + \varphi_R) - \eta_1\varphi_D$$

When η_1 and η_2 are both zero, the model reduces to the static EBCM. When edge turnover is fast, the network approaches mean-field mixing. This framework can model partnership dynamics, workplace contacts, or serosorting — where contact patterns change based on disease status.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
```

We define all symbolic parameters used throughout this vignette.

```
@parameters β γ κ η₁ η₂ σ
pgf = poisson_pgf(κ)
```

```
DegreePGF(z, exp((-1 + z)*κ))
```

We also define a helper function to extract population-level trajectories from an ODE solution, given a Poisson PGF.

```
function extract_SIR(sol, result, κ_val)
    ψ(x) = exp(κ_val * (x - 1))
    θ_vals = compartment(sol, result, :θ)
    S = ψ.(θ_vals)
    R = compartment(sol, result, :R)
    I = 1.0 .- S .- R
    return S, I, R
end
```

```
extract_SIR (generic function with 1 method)
```

Static SIR Baseline

Before introducing edge dynamics, we build a standard static SIR on a Poisson network with mean degree $\kappa = 5$.

[!NOTE]

$R_0 = 2$ anchor. We use $\gamma = 0.25$, $\kappa = 5$, $T = 2/5$, and per-edge $\beta = 1/6$ with 1% initial infection. Dynamic variants reuse these disease parameters so changes reflect edge turnover.

```
static_result = build_sir(pgf,  $\beta$ ,  $\gamma$ ; form=:expanded)
 $\gamma_{\text{val}}$  = 0.25
 $R0_{\text{target}}$  = 2.0
seed_fraction = 0.01
 $\kappa_{\text{val}}$  = 5.0
 $T_{\text{val}}$  =  $R0_{\text{target}}$  /  $\kappa_{\text{val}}$ 
 $\beta_{\text{val}}$  =  $T_{\text{val}}$  *  $\gamma_{\text{val}}$  / (1 -  $T_{\text{val}}$ )
```

0.16666666666666669

```
ic_static = default_initial_conditions(static_result; seed_fraction = seed_fraction)
p_static = Dict( $\beta \Rightarrow \beta_{\text{val}}$ ,  $\gamma \Rightarrow \gamma_{\text{val}}$ ,  $\kappa \Rightarrow \kappa_{\text{val}}$ )
tspan = (0.0, 40.0)
prob_static = ODEProblem(static_result.system, merge(ic_static, p_static), tspan)
sol_static = solve(prob_static, Tsit5())
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 19-element Vector{Float64}:

0.0
0.09094961414630146
0.4737685765271724
1.1194667152434108
1.943336414785521
3.0120992575199015
4.3439876075151584
6.05162312862639
8.038814788576333
10.264632085858668
12.902851341979162
15.66880091646387

```

18.811957922393077
22.414052919409702
25.809916966093233
29.309945524416857
32.83684495305802
36.47307124210958
40.0
u: 19-element Vector{Vector{Float64}}:
 [0.0, 0.01, 0.0, 0.010000000000000009, 1.0]
 [0.00023337834911741072, 0.010530760492159199, 0.00023164663522085038, 0.01037806111590853,
 [0.0013544160177506501, 0.01294725683909643, 0.001306377474706858, 0.012124377065668991, 0.
 [0.00382160921666847, 0.01779710582741937, 0.0035416461114554324, 0.015715971524995466, 0.9
 [0.008268158783602953, 0.025750307125295316, 0.007368067176027353, 0.021738353948852693, 0.
 [0.0169315070644167, 0.039915145657425155, 0.014542814343354757, 0.032608628816250616, 0.99
 [0.034198855349653653, 0.06539292529465035, 0.028456992743610635, 0.05216345940495296, 0.98
 [0.07156004503309873, 0.11210297116402208, 0.05786323287091843, 0.0872242947455901, 0.96142
 [0.14329259869188998, 0.17653829549710018, 0.11260914561193848, 0.13214898483575935, 0.9249
 [0.25738218473111085, 0.22623468104112895, 0.19525422137144569, 0.158193163486497, 0.869830
 [0.40863287482711014, 0.22152758723135418, 0.29539037360385684, 0.13784317271870297, 0.8030
 [0.5455176684818529, 0.17071879580911797, 0.3748730277643796, 0.09144808468367156, 0.750084
 [0.6543987751513454, 0.10836195962009218, 0.42859458683611323, 0.048436423377915525, 0.7142
 [0.7273715284102072, 0.057840790305424654, 0.4584244270936047, 0.02117160689295743, 0.69438
 [0.7636170366723547, 0.030210903469530336, 0.47070750486957164, 0.009315432025932407, 0.680
 [0.7826292310256151, 0.014919276811946247, 0.4761712214885133, 0.0039298053567058564, 0.682
 [0.791955844161402, 0.007143850397691966, 0.4784788195388369, 0.0016349953276990905, 0.6810
 [0.7964738721524173, 0.00327890471864087, 0.47945568860335835, 0.00065996253212765, 0.68030
 [0.7984922710955112, 0.001518449398545614, 0.4798423903459016, 0.0002734032508876271, 0.680

```

```

S_s, I_s, R_s = extract_SIR(sol_static, static_result, 5.0)
plot(sol_static.t, S_s, label="S", lw=2, color=:blue)
plot!(sol_static.t, I_s, label="I", lw=2, color=:red)
plot!(sol_static.t, R_s, label="R", lw=2, color=:green)
xlabel!("Time")
ylabel!("Fraction of population")
title!("Static SIR ( $\kappa=5$ ,  $R_0=2$ ,  $\gamma=0.25$ )")

```

Dynamic SIR Model

Now we build an SIR model on a dynamic network where edges break at rate $\eta_2 = 0.3$ and form at rate $\eta_1 = 0.5$. At equilibrium (before the epidemic), the fraction of dormant edges would be

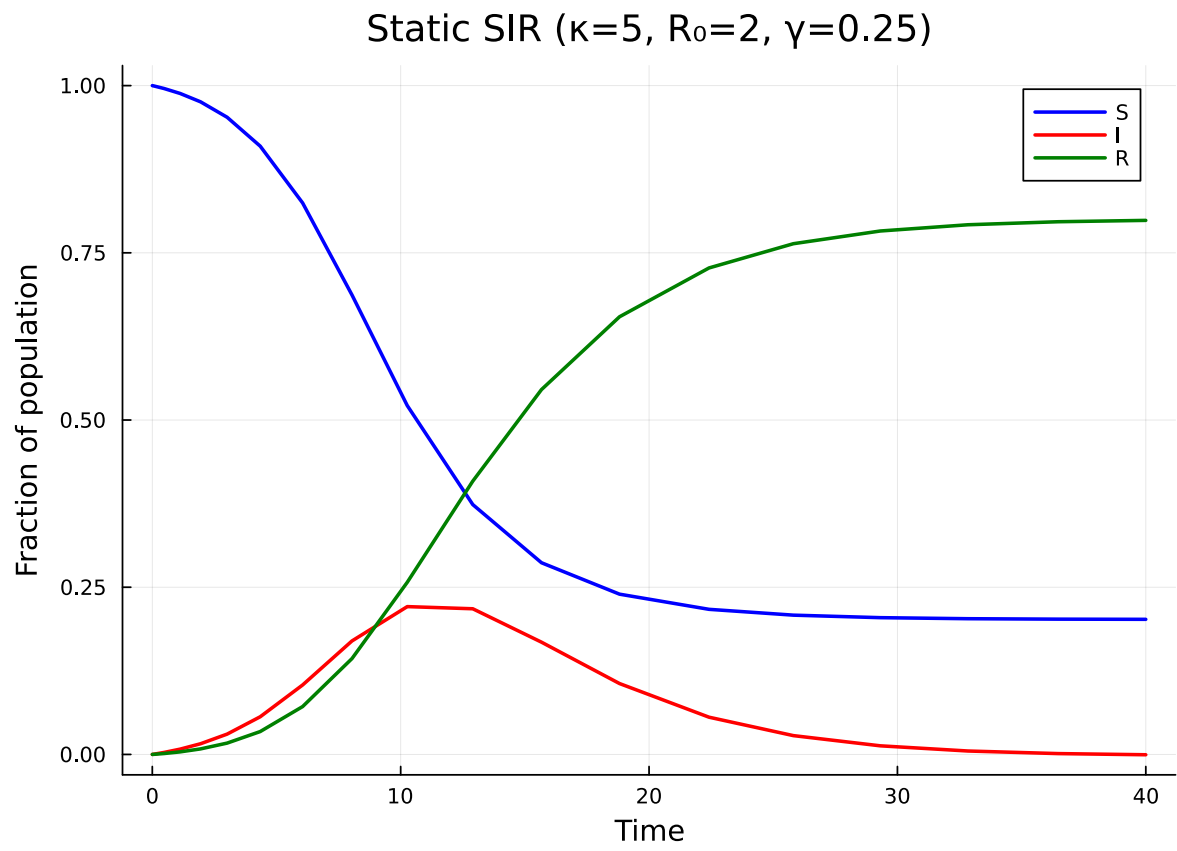


Figure 1: Figure 1: SIR epidemic on a static Poisson network with mean degree $\kappa = 5$.

$\eta_2/\eta_1 = 0.6$. Since we start with all edges active ($\varphi_D(0) = 0$), the dormant population builds up as edges break.

```
progression = DiseaseProgression(
    [DiseaseStage(:I; transmission_rate=β), DiseaseStage(:R)],
    [DiseaseTransition(:I, :R, γ)];
    entry=:I
)
dyn_model = DynamicConfigurationModel(pgf, progression, η1, η2)
dyn_result = build_edge_system(dyn_model)
```

EdgeModelSystem(Model dynamic_ebm:

Equations (6):

6 standard: see equations(dynamic_ebm)

Unknowns (6): see unknowns(dynamic_ebm)

pop_R(t)

pop_I(t)

φ_R(t)

φ_I(t)

?

Parameters (6): see parameters(dynamic_ebm)

κ

ρ

η₂

β

?

Observed (3): see observed(dynamic_ebm), Dict{Symbol, Any}(:φ_D => φ_D(t), :R => pop_R(t), :φ_1 + θ(t))*κ)), :rho_param => ρ, :edge_seed_groups => Any[(entry = φ_I(t), theta = θ(t), phi_S_1 + θ(t))*κ)*(1 - ρ)) / exp(0))]))

The dynamic SIR model has 5 ODEs — the standard θ , φ_I , φ_R , R plus the dormant edge state φ_D :

```
eqs = ModelingToolkit.equations(dyn_result.system)
println("Number of ODEs: ", length(eqs))
for eq in eqs
    println(eq)
end
```

Number of ODEs: 6

Differential(t, 1)(pop_R(t)) ~ pop_I(t)*γ

Differential(t, 1)(pop_I(t)) ~ -pop_I(t)*γ + φ_I(t)*exp((-1 + θ(t))*κ)*β*κ*(1 - ρ)

```

Differential(t, 1)(φ_R(t)) ~ φ_I(t)*γ - φ_R(t)*η2 + φ_D(t)*pop_R(t)*η1,
Differential(t, 1)(φ_I(t)) ~ (φ_I(t)*exp(0)*(β + γ + η2) - pop_I(t)*exp(0)*φ_D(t)*η1 -
φ_I(t)*exp((-1 + θ(t))*κ)*β*κ*(1 - ρ)) / (-exp(0))
Differential(t, 1)(φ_D(t)) ~ (φ_I(t) + φ_S(t) + φ_R(t))*η2 - φ_D(t)*η1,
Differential(t, 1)(θ(t)) ~ -φ_I(t)*β - (φ_I(t) + φ_S(t) + φ_R(t))*η2 + φ_D(t)*η1,

```

We can also inspect the state variables and observables:

```

println("State variables: ", keys(dyn_result.variables))
println("Observables: ", keys(dyn_result.observables))

```

```

State variables: [:φ_D, :R, :φ_I, :pop_I, :pop_R, :φ_R, :θ]
Observables: [:I, :φ_S, :S, :edge_hazard, :excess_hazard]

```

Now solve the dynamic model:

```

ic_dyn = default_initial_conditions(dyn_result; seed_fraction = seed_fraction)
p_dyn = Dict{β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η1 ⇒ 0.5, η2 ⇒ 0.3}
prob_dyn = ODEProblem(dyn_result.system, merge(ic_dyn, p_dyn), tspan)
sol_dyn = solve(prob_dyn, Tsit5())

```

```

retcode: Success
Interpolation: specialized 4th order "free" interpolation
t: 23-element Vector{Float64}:
 0.0
 0.005426802233448098
 0.07623338948392797
 0.2368716258384217
 0.4585606033643804
 0.7500161282253746
 1.1286286449133054
 1.607001832246569
 2.2017864493186488
 2.9286792727618853
 ⋮
 9.565754242707344
11.777177543376528
14.397670765121521
17.52417251643269
21.27470058210549
25.688358055893378

```

```

30.687619817862505
36.03710015387654
40.0
u: 23-element Vector{Vector{Float64}}:
 [0.0, 0.01, 0.0, 0.010000000000000009, 0.0, 1.0]
 [1.3588086731059805e-5, 0.010031013798094185, 1.355990182872345e-5, 0.010005720629467663, 0.0100019453238290120816, 0.010403007406963538, 0.0001890684984721557, 0.010053744787657993, 0.000626284460240841, 0.011060667938833087, 0.0005756446256386184, 0.010020939155518441, 0.00012573010698518135, 0.011661707636353753, 0.0010780416015207265, 0.009775573396525445, 0.002124850860954767, 0.012097555932978224, 0.0016800296493909453, 0.009281788504503657, 0.0032816823442852927, 0.012291674610161302, 0.00236716127232518, 0.008545531238987328, 0.004748263438237318, 0.01218557700716699, 0.00310223947647874, 0.007629732173507355, 0.17006531225454865407, 0.011756793706305507, 0.003850979063331088, 0.006618622552238603, 0.008602807423858178, 0.01101727107150938, 0.004582007429281046, 0.005599666101226999, 0.2021012280895385062, 0.004558060622212068, 0.007771068909757127, 0.001698982406445426, 0.02316572638155159, 0.0032973238456388628, 0.008316017541665706, 0.0012079686268441727, 0.024957048831204802, 0.0022382698285236323, 0.008797350968891738, 0.0008125882934927693, 0.026356623679105896, 0.0014065586863688526, 0.009195396291639292, 0.0005084028964489034, 0.027367131122695572, 0.0008044717136025699, 0.009496322456985402, 0.00029018443963686736, 0.028017440120538015, 0.0004164776673186457, 0.00969644788192133, 0.00015008801743582586, 0.02838423586370554, 0.00019748438565136172, 0.00981193726395056, 7.11381212463379e-5, 0.20723373644087925, 0.7836098463374024]
 [0.028566122836071538, 8.884806973556433e-5, 0.009874855960717535, 3.200297089006768e-5, 0.20696139588972604, 0.7838385174060296]
 [0.028632569550637314, 4.9155392162173496e-5, 0.00990278521728844, 1.771018007131673e-5, 0.2066499075021118, 0.7841340551921787]

```

```

_, I_static, _ = extract_SIR(sol_static, static_result, 5.0)
_, I_dynamic, _ = extract_SIR(sol_dyn, dyn_result, 5.0)

plot(sol_static.t, I_static, label="Static", lw=2, linestyle=:dash, color=:grey)
plot!(sol_dyn.t, I_dynamic, label="Dynamic ( $\eta_1=0.5$ ,  $\eta_2=0.3$ )", lw=2, color=:red)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Static vs Dynamic Network")

```

Edge dynamics alter the epidemic because edges breaking during an epidemic can interrupt transmission chains, while new edges forming can create new pathways for infection.

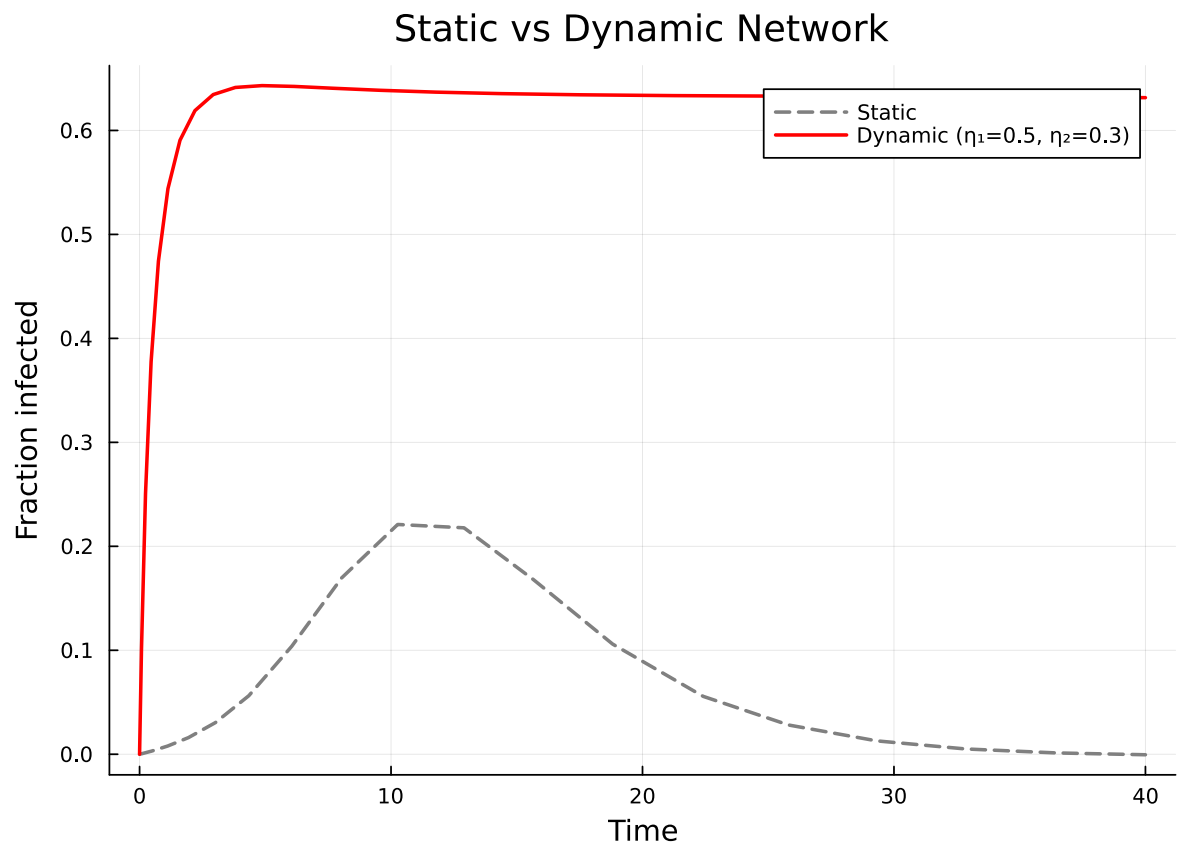


Figure 2: Comparison of the infected fraction $I(t)$ on static versus dynamic networks.

Effect of Edge Dynamics

We now compare three scenarios to see how the rate of edge turnover affects the epidemic:

1. Static network — no edge dynamics ($\eta_1 = \eta_2 = 0$)
2. Slow turnover — $\eta_1 = 0.1, \eta_2 = 0.05$
3. Fast turnover — $\eta_1 = 2.0, \eta_2 = 1.2$

```
# Slow edge dynamics
p_slow = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η₁ ⇒ 0.1, η₂ ⇒ 0.05)
prob_slow = ODEProblem(dyn_result.system, merge(ic_dyn, p_slow), tspan)
sol_slow = solve(prob_slow, Tsit5())

# Fast edge dynamics
p_fast = Dict(β ⇒ β_val, γ ⇒ γ_val, κ ⇒ κ_val, η₁ ⇒ 2.0, η₂ ⇒ 1.2)
prob_fast = ODEProblem(dyn_result.system, merge(ic_dyn, p_fast), tspan)
sol_fast = solve(prob_fast, Tsit5())
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 58-element Vector{Float64}:

```
0.0
0.0013567906468560595
0.02469576420144653
0.07217164063790674
0.13476189754412576
0.21691564514956552
0.3217498992912283
0.4532120202114286
0.6149203910068808
0.8114490682700117
```

⌈

```
33.579521130915296
34.447550365264114
35.315583694816944
36.18362195320621
37.05166456487838
37.91971039190691
38.78775839559056
39.65580795402915
```

40.0

u: 58-element Vector{Vector{Float64}}:

```
[0.0, 0.01, 0.0, 0.010000000000000009, 0.0, 1.0]
[3.393293789294786e-6, 0.010007748904225962, 3.3873990657156127e-6, 0.009989241231924314, 0.009989241231924314, 0.009989241231924314]
[6.21443750007653e-5, 0.01012618025756914, 6.025408099536631e-5, 0.009799975734591721, 0.009799975734591721, 0.009799975734591721]
[0.00018344239819344482, 0.010301003696268506, 0.0001682766187724311, 0.009405893558505197, 0.009405893558505197, 0.009405893558505197]
[0.0003458494380158382, 0.010444276944816273, 0.00029685585780044933, 0.008899292403627383, 0.008899292403627383, 0.008899292403627383]
[0.0005615089390823911, 0.010543431644144707, 0.00044571965559339117, 0.008285961918872372, 0.008285961918872372, 0.008285961918872372]
[0.0008385442943921983, 0.010585461255872945, 0.0006096037862432908, 0.007599687751123587, 0.007599687751123587, 0.007599687751123587]
[0.0011862326740240219, 0.010562201880981179, 0.0007838482075202474, 0.006878010918856361, 0.006878010918856361, 0.006878010918856361]
[0.0016115158128362564, 0.01046834019683561, 0.0009638092581605198, 0.0061617302893692966, 0.0061617302893692966, 0.0061617302893692966]
[0.002121895405625661, 0.010300072975685448, 0.0011476663735937064, 0.005482151345442406, 0.005482151345442406, 0.005482151345442406]
[0.019887265923763862, 7.924701436625562e-5, 0.006904746725246053, 2.790830802525735e-5, 0.20813859177484437, 0.7864682788846127]
[0.01990338958341707, 6.956061469501282e-5, 0.006910282703788183, 2.449685017966291e-5, 0.2081372657868672, 0.7864658201891598]
[0.019917542491068606, 6.105804680607673e-5, 0.006915141970381801, 2.150237535919138e-5, 0.2081361017358969, 0.7864636621754992]
[0.019929965510186224, 5.359464936963747e-5, 0.006919407264339365, 1.8873918121855395e-5, 0.2081350786672871, 0.7864617692512406]
[0.01994087005231088, 4.704344389317875e-5, 0.0069231511875529035, 1.6566745352697047e-5, 0.20813417936278675, 0.7864601090033885]
[0.019950441692495314, 4.129296535010092e-5, 0.006926437458417932, 1.4541591124708468e-5, 0.2081333892606816, 0.7864586524289144]
[0.01995884333366253, 3.6245363174317194e-5, 0.006929322008142755, 1.2763987027747716e-5, 0.2081326955975449, 0.7864573740497522]
[0.0199662179774672, 3.1814738173060015e-5, 0.006931853937195431, 1.1203675777488075e-5, 0.20813208690006885, 0.7864562517701954]
[0.019968886036807843, 3.0212225998847652e-5, 0.006931482967062298, 1.0637617358936704e-5, 0.20822302602778925, 0.7863646863265598]
```

```
_, I_static, _ = extract_SIR(sol_static, static_result, 5.0)
_, I_slow, _ = extract_SIR(sol_slow, dyn_result, 5.0)
_, I_fast, _ = extract_SIR(sol_fast, dyn_result, 5.0)

plot(sol_static.t, I_static, label="Static", lw=2, color=:black, linestyle=:dash)
plot(sol_slow.t, I_slow, label="Slow ( $\eta_1=0.1$ ,  $\eta_2=0.05$ )", lw=2, color=:blue)
plot(sol_dyn.t, I_dynamic, label="Medium ( $\eta_1=0.5$ ,  $\eta_2=0.3$ )", lw=2, color=:orange)
plot(sol_fast.t, I_fast, label="Fast ( $\eta_1=2.0$ ,  $\eta_2=1.2$ )", lw=2, color=:red)
xlabel!("Time")
ylabel!("Fraction infected")
title!("Effect of Edge Turnover on Epidemic Dynamics")
```

The plot shows that edge dynamics modify both the peak and timing of the epidemic. The ratio

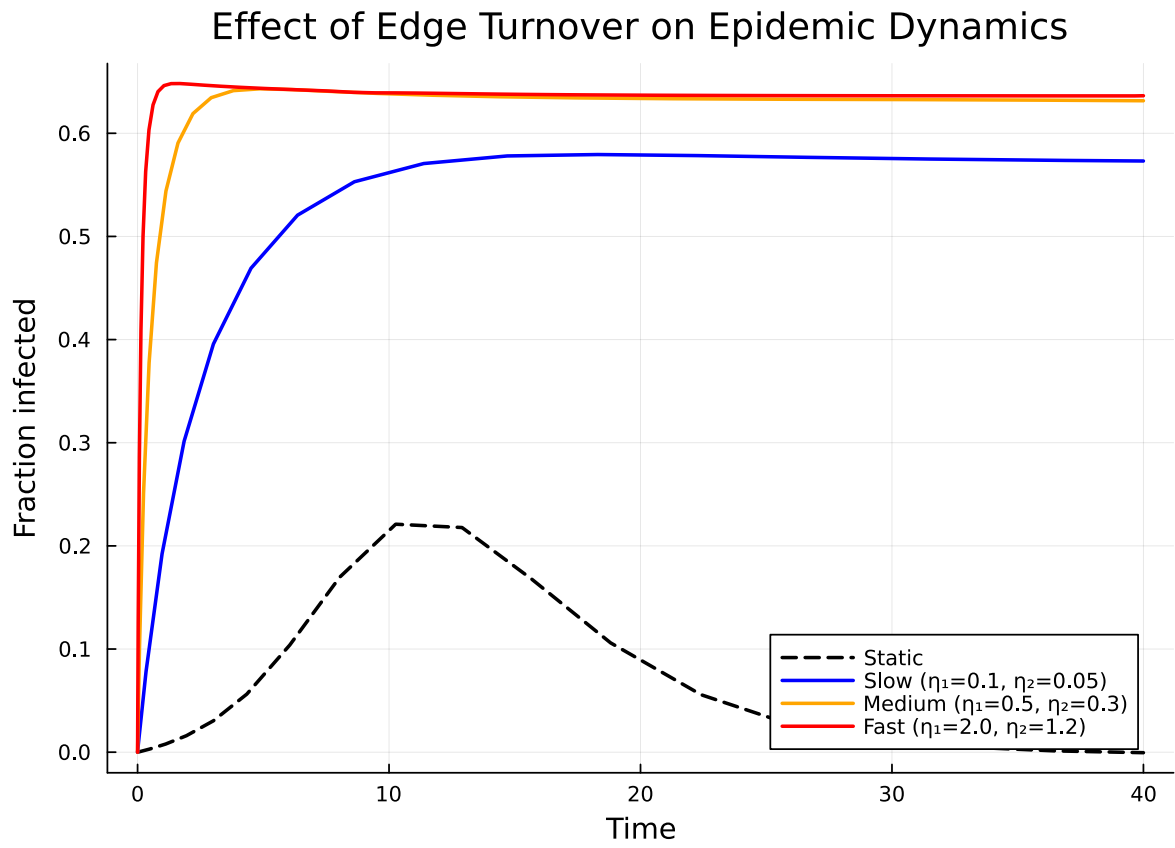


Figure 3: Figure 3: Effect of edge turnover rate on the epidemic curve. Faster edge dynamics alter the timing and magnitude of the peak.

η_2/η_1 controls the equilibrium fraction of dormant edges, while the absolute magnitudes of η_1 and η_2 control how quickly the network responds to population-level changes.

Inspecting Edge States

The dynamic model tracks four types of edge stubs: susceptible (φ_S), infectious (φ_I), recovered (φ_R), and dormant (φ_D). The susceptible fraction is computed algebraically as $\varphi_S = \psi'(\theta)/\psi'(1)$, while the others are ODE states.

```
κ_val = 5.0
θ_vals = compartment(sol_dyn, dyn_result, :θ)
φ_S_vals = exp.(κ_val .* (θ_vals .- 1.0))
φ_I_vals = compartment(sol_dyn, dyn_result, :φ_I)
φ_R_vals = compartment(sol_dyn, dyn_result, :φ_R)
φ_D_vals = compartment(sol_dyn, dyn_result, :φ_D)

plot(sol_dyn.t, φ_S_vals, label="φ_S (susceptible)", lw=2, color=:blue)
plot!(sol_dyn.t, φ_I_vals, label="φ_I (infectious)", lw=2, color=:red)
plot!(sol_dyn.t, φ_R_vals, label="φ_R (recovered)", lw=2, color=:green)
plot!(sol_dyn.t, φ_D_vals, label="φ_D (dormant)", lw=2, color=:purple)
xlabel!("Time")
ylabel!("Edge stub fraction")
title!("Edge State Dynamics (η1=0.5, η2=0.3)")
```

Initially, all edges are active and connected to susceptible neighbours ($\varphi_S \approx 1$). As the epidemic progresses, φ_S decreases while φ_I and φ_R grow. The dormant edge fraction φ_D builds up from zero as active edges break, eventually reaching an equilibrium determined by the ratio η_2/η_1 .

Dynamic SEIR

The dynamic network framework also works with complex disease progressions. Here we build a dynamic SEIR model, which adds an exposed (latent) stage E that is non-infectious.

```
seir_progression = DiseaseProgression(
    [
        DiseaseStage(:E; transmission_rate=0),
        DiseaseStage(:I; transmission_rate=β),
        DiseaseStage(:R)
    ],
    [
```

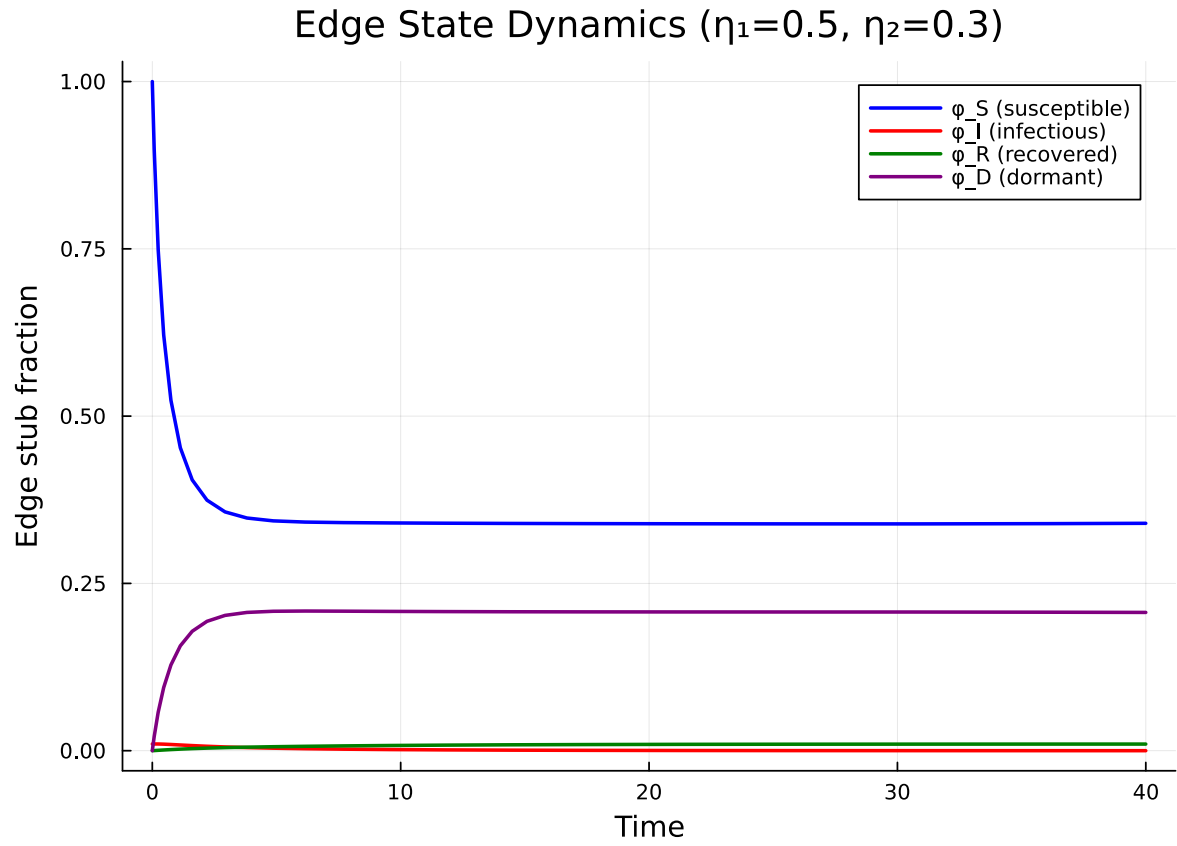


Figure 4: Figure 4: Edge-level state dynamics during the epidemic on a dynamic network ($\eta_1=0.5$, $\eta_2=0.3$). The dormant edge fraction (ϕ_D) grows as active edges break.

```

        DiseaseTransition(:E, :I,  $\sigma$ ),
        DiseaseTransition(:I, :R,  $\gamma$ )
    ];
    entry=:E
)
dyn_seir_model = DynamicConfigurationModel(pgf, seir_progression,  $\eta_1$ ,  $\eta_2$ )
dyn_seir_result = build_edge_system(dyn_seir_model)

```

EdgeModelSystem(Model dynamic_ebm:

Equations (8):

8 standard: see equations(dynamic_ebm)

Unknowns (8): see unknowns(dynamic_ebm)

pop_R(t)

pop_I(t)

pop_E(t)

$\varphi_R(t)$

?

Parameters (7): see parameters(dynamic_ebm)

κ

ρ

η_2

β

?

Observed (3): see observed(dynamic_ebm), Dict{Symbol, Any}(: φ_D => $\varphi_D(t)$, :R => pop_R(t), : $\varphi_{1 + \theta(t)}\kappa$)), :rho_param => ρ , :edge_seed_groups => Any[(entry = $\varphi_E(t)$, theta = $\theta(t)$, phi_S_1 + $\theta(t)$)* κ)*(1 - ρ) / exp(0))]))

The dynamic SEIR model has 6 ODEs — adding φ_E to the dynamic SIR's set:

```

seir_eqs = ModelingToolkit.equations(dyn_seir_result.system)
println("Number of ODEs: ", length(seir_eqs))
for eq in seir_eqs
    println(eq)
end

```

Number of ODEs: 8

Differential(t, 1)(pop_R(t)) ~ pop_I(t)* γ

Differential(t, 1)(pop_I(t)) ~ -pop_I(t)* γ + pop_E(t)* σ

Differential(t, 1)(pop_E(t)) ~ -pop_E(t)* σ + $\varphi_I(t)$ *exp((-1 + $\theta(t)$)* κ)* β * κ *(1 - ρ)

Differential(t, 1)($\varphi_R(t)$) ~ $\varphi_I(t)$ * γ - $\varphi_R(t)$ * η_2 + $\varphi_D(t)$ *pop_R(t)* η_1

```

Differential(t, 1)(φ_I(t)) ~ -φ_I(t)*(β + γ + η2) + φ_E(t)*σ + pop_I(t)*φ_D(t)*η1,
Differential(t, 1)(φ_E(t)) ~ (φ_E(t)*exp(θ)*(η2 + σ) - pop_E(t)*exp(θ)*φ_D(t)*η1 -
φ_I(t)*exp((-1 + θ(t))*κ)*β*κ*(1 - ρ)) / (-exp(θ))
Differential(t, 1)(φ_D(t)) ~ (φ_I(t) + φ_E(t) + φ_S(t) + φ_R(t))*η2 - φ_D(t)*η1,
Differential(t, 1)(θ(t)) ~ -φ_I(t)*β - (φ_I(t) + φ_E(t) + φ_S(t) + φ_R(t))*η2 + φ_D(t)*η1,

```

```
println("State variables: ", keys(dyn_seir_result.variables))
```

```
State variables: [:φ_D, :R, :φ_I, :pop_I, :pop_R, :φ_R, :θ, :φ_E, :pop_E]
```

The dynamic framework is fully composable — any disease progression supported by `EdgeBased-Models.jl` can be combined with edge formation and breaking dynamics, simply by wrapping it in a `DynamicConfigurationModel` instead of a `StaticConfigurationModel`.

Simulation validation

This vignette focuses on a scenario for which `NetworkOutbreaks.jl` does not yet provide an out-of-the-box stochastic ground truth (multi-type host graphs with prescribed mixing matrices, time-varying networks via the EBCM rewiring schedule, multiplex layers, degree-correlated configuration models, clustered graphs with prescribed triangle counts, or final-size sweeps over R_0).

A future revision will add the missing primitives to `NetworkOutbreaks.jl` and overlay the corresponding Gillespie SSA ribbon here. Until then, see [vignette 01](#) for the validation pattern on a single-layer Poisson configuration-model network with the same canonical parameters.